PREPROCESADORES CSS: SASS Y SCSS

1. PREPROCESADORES DE CSS

A medida que una hoja de estilos crece, mantener colores, tamaños, componentes y reglas repetidas puede resultar difícil. Los **preprocesadores** permiten **escribir estilos con una sintaxis más potente y organizada y transformarlos después en CSS** estándar que el navegador puede interpretar.

En este tema se estudiará principalmente **Sass** utilizando su sintaxis SCSS, por ser compatible con la sintaxis de CSS y la opción más habitual para incorporarlo a proyectos existentes.

1.1. Qué es un preprocesador CSS

Un preprocesador CSS es una herramienta que recibe un archivo fuente escrito con funcionalidades adicionales —variables, anidación, mixins, funciones o estructuras de control— y genera como resultado una hoja CSS convencional.

```
archivo.scss → compilador Sass → archivo.css → navegador
```

El navegador no interpreta directamente los archivos .scss o .sass. Por eso, antes de publicar la web es necesario compilarlos a CSS.

IDEA CLAVE. Sass no sustituye a CSS. Es una capa de trabajo que ayuda a escribir y organizar CSS; el resultado final sigue siendo CSS.

1.2. Preprocesadores y posprocesadores

Un preprocesador actúa antes de obtener el CSS final. Un **posprocesador** trabaja sobre CSS ya generado para transformarlo, optimizarlo o adaptarlo. PostCSS es una herramienta de transformación de CSS mediante plugins y no debe confundirse con Sass.

Herramienta	Momento	Ejemplos de uso
Sass / SCSS	Antes del CSS final	Variables, mixins, módulos, funciones, anidación.
PostCSS	Sobre CSS	Autoprefixer, transformaciones y optimizaciones mediante plugins.

1.3. Sass, SCSS y otras alternativas

Sass (Syntactically Awesome Style Sheets) es un lenguaje de hojas de estilo que se compila a CSS. Actualmente la implementación de referencia es Dart Sass.

Sass dispone de dos sintaxis. Ambas ofrecen las mismas capacidades, pero escriben el código de forma diferente:

Sintaxis	Extensión	Características
SCSS	.scss	Utiliza llaves y punto y coma. Todo CSS válido es también SCSS válido.
Sintaxis indentada Sass	.sass	Utiliza indentación y omite llaves y punto y coma.

```scss
// SCSS
$color: #3498db;

.boton {
  background-color: $color;
}
```

```sass
//.SASS
$color: #3498db

.boton
  background-color: $color
```

LESS es otro preprocesador conocido y utiliza una sintaxis similar a CSS. En este tema se trabajará con SCSS para concentrar el aprendizaje en un único entorno y porque permite pasar de CSS a Sass de forma progresiva.

2. INSTALACIÓN Y COMPILACIÓN DE SASS

2.1. Dart Sass

La implementación recomendada para proyectos actuales es Dart Sass. Puede utilizarse desde la línea de comandos y también está integrada en numerosas herramientas de desarrollo.

Si tenemos Node.js instalado, una forma sencilla de disponer del compilador es instalar el paquete sass:

```
npm install -g sass
sass --version
```

También puede ejecutarse sin instalación global mediante npx:

```
npx sass estilos.scss estilos.css
```

2.2. Compilar un archivo SCSS

La compilación básica recibe un archivo SCSS de entrada y genera un archivo CSS de salida:

```
sass scss/estilos.scss css/estilos.css
```

Durante el desarrollo resulta útil mantener el compilador observando los cambios. Cada vez que se guarde el archivo SCSS, Sass actualizará automáticamente el CSS:

```
sass --watch scss/estilos.scss:css/estilos.css
```

También puede observarse una carpeta completa:

```
sass --watch scss:css
```

2.3. Salida expandida y comprimida

Durante el desarrollo suele interesar un CSS legible. Para producción puede generarse una versión comprimida:

```
sass scss/estilos.scss css/estilos.css --style=expanded
sass scss/estilos.scss css/estilos.min.css --style=compressed
```

- **Expanded**: genera un archivo CSS organizado, con saltos de línea, indentación y espacios. **Ventaja**: es fácil de leer, revisar y depurar.

- **Compressed**: genera el mismo CSS, pero elimina los espacios y saltos de línea innecesarios. **Ventaja**: ocupa menos espacio y reduce el tamaño del archivo que debe descargar el navegador.

QUÉ CAMBIA Y QUÉ NO

Los dos archivos representan los mismos estilos: cambia la presentación del código, no el resultado visual. Para revisar errores utiliza expanded; para publicar una versión más ligera puedes utilizar compressed. El archivo .min.css no se edita a mano: se vuelve a generar a partir del SCSS.

3. VARIABLES

Las variables Sass permiten **guardar un valor y reutilizarlo en diferentes reglas**. Se identifican mediante el símbolo **$**. Son útiles para centralizar colores, medidas, tipografías y otros valores repetidos.

```
$color-primario: #3498db;
$color-secundario: #2ecc71;
$fuente-principal: Arial, sans-serif;
$espaciado: 1rem;
```

```
body {
  font-family: $fuente-principal;
  color: $color-primario;
}

.boton {
  background-color: $color-secundario;
  padding: $espaciado;
}
```

Las variables Sass existen durante la compilación. No son lo mismo que las propiedades personalizadas de CSS (--color), que permanecen en el CSS final y pueden cambiar en tiempo de ejecución.

Sass	CSS
$color: #3498db;	--color: #3498db;
Se resuelve al compilar.	Permanece en el CSS y la interpreta el navegador.
Se usa como $color.	Se usa como var(--color).

3.1. Ámbito de las variables

Una variable declarada en el nivel superior pertenece al módulo. Una variable declarada dentro de un bloque suele ser local a ese bloque. Esta diferencia evita que variables internas modifiquen accidentalmente otras partes del proyecto.

```
$color: #333;

.tarjeta {
  $fondo: #f5f5f5;
  color: $color;
  background: $fondo;
}

// $fondo no está disponible aquí.
```

EXPLICACIÓN

Las variables de nivel superior pueden utilizarse en el módulo donde se declaran. Una variable creada dentro de una regla se utiliza dentro de ese bloque, pero no debe darse por disponible fuera de él. Esta distinción evita dependencias difíciles de localizar.

4. OPERADORES Y CÁLCULOS

Sass permite realizar operaciones matemáticas y evaluar condiciones en archivos SCSS. Estos cálculos se resuelven durante la compilación: el CSS final contiene el resultado. Resultan útiles para tamaños, márgenes, espaciados y estilos configurables.

4.1. Operadores aritméticos

Operador	Operación	Ejemplo	Resultado
+	Suma	20px + 10px	30px
–	Resta	100px - 25px	75px
*****	Multiplicación	10px * 3	30px
math.div()	División	math.div(100px, 4)	25px
%	Resto	10 % 3	1

SCSS:

```scss
@use "sass:math";
$espaciado: 10px;
$ancho: 800px;
.caja {
  padding: $espaciado * 2;
  margin: $espaciado + 5px;
  width: $ancho - 100px;
  height: math.div(200px, 2);
}
```

CSS GENERADO:

```css
.caja {
  padding: 20px;
  margin: 15px;
  width: 700px;
  height: 100px;
}
```

Si $espaciado pasa de 10px a 15px, Sass recalcula automáticamente las propiedades que utilizan esa variable.

4.2. División con math.div()

La división matemática se realiza con **math.div(),** del módulo incorporado **sass:math**. Se carga al comienzo del archivo mediante @use.

SCSS:

```
@use "sass:math";
.contenedor { width: math.div(900px, 3); }
.columna { width: math.div(100%, 4); }
.titulo { font-size: math.div(48px, 2); }
```

CSS GENERADO:

```
.contenedor { width: 300px; }
.columna { width: 25%; }
.titulo { font-size: 24px; }
```

4.3. Operadores de comparación

Comparan dos valores y devuelven true (verdadero) o false (falso). Se emplean principalmente con @if y @else.

Operador	Significado	Ejemplo
==	Igual	$ancho == 800px
!=	Distinto	$ancho != 800px
>	Mayor	$ancho > 600px
<	Menor	$ancho < 600px
>=	Mayor o igual	$ancho >= 600px
<=	Menor o igual	$ancho <= 600px

SCSS:

```
$ancho: 800px;
.contenedor {
  @if $ancho > 600px { width: 80%; }
  @else { width: 100%; }
}
```

CSS GENERADO:

```
.contenedor { width: 80%; }
```

La condición es verdadera al compilar. No cambia automáticamente cuando el usuario redimensiona la ventana; para ello se utilizan media queries.

4.4. Operadores lógicos

Operador	Significado
and	Deben cumplirse ambas condiciones
or	Debe cumplirse al menos una condición
not	Invierte el valor lógico

SCSS con and:

```
$ancho: 800px;
.contenedor {
  @if $ancho >= 600px and $ancho <= 1000px {
    padding: 30px;
  } @else { padding: 10px; }
}
```

CSS GENERADO: `.contenedor { padding: 30px; }`. Ambas comparaciones son verdaderas.

SCSS con or:

```
$tema: oscuro;
body {
  @if $tema == oscuro or $tema == noche {
    background-color: black;
    color: white;
  } @else {
    background-color: white;
    color: black;
  }
}
```

CSS GENERADO: `body { background-color: black; color: white; }`.

4.5. Operador resto (%)

Devuelve el resto de una división. Es útil, por ejemplo, para comprobar si un número es par.

SCSS:

```
$numero: 6;
.elemento {
  @if $numero % 2 == 0 {
    background-color: lightblue;
  } @else {
    background-color: lightcoral;
  }
}
```

CSS GENERADO: `.elemento { background-color: lightblue; }`, porque el resto de dividir 6 entre 2 es cero.

4.6. Operaciones con unidades y calc()

Sass opera con unidades compatibles: 100px + 50px da 150px; 10px * 3 da 30px; 2rem + 1rem da 3rem. No puede resolver directamente una resta como 100% - 20px, porque depende de una dimensión que conocerá el navegador.

```
.caja { width: calc(100% - 20px); }
```

En este caso, calc() es una función nativa de CSS y el cálculo se realiza en el navegador con las dimensiones reales disponibles.

5. ANIDACIÓN DE SELECTORES

SCSS permite escribir selectores dentro de otros selectores. Esto hace visible la relación entre los elementos y evita repetir parte del selector.

SCSS:

```
.tarjeta {
  background-color: lightgray;
  padding: 20px;
  border-radius: 10px;

  h2 {
    color: navy;
    font-size: 24px;
  }

  p {
    color: darkslategray;
    font-size: 16px;
  }

  a {
    color: blue;
    text-decoration: none;

    &:hover {
      color: red;
      text-decoration: underline;
    }
```

```
  }
}
```

CSS GENERADO

```
.tarjeta {
  background-color: lightgray;
  padding: 20px;
  border-radius: 10px;
}

.tarjeta h2 {
  color: navy;
  font-size: 24px;
}

.tarjeta p {
  color: darkslategray;
  font-size: 16px;
}

.tarjeta a {
  color: blue;
  text-decoration: none;
}

.tarjeta a:hover {
  color: red;
  text-decoration: underline;
}
```

¿Qué está ocurriendo?

- h2 dentro de .tarjeta se convierte en .tarjeta h2.
- p dentro de .tarjeta se convierte en .tarjeta p.
- a dentro de .tarjeta se convierte en .tarjeta a.
- &:hover dentro de a se convierte en .tarjeta a:hover.

Importante: anidar un selector dentro de otro genera, por defecto, un selector descendiente, no necesariamente un hijo directo.

SCSS:

```
.tarjeta {
  > h2 {
    color: navy;
  }
}
```

CSS GENERADO

```
.tarjeta > h2 {
 color: navy;
}
```

En SCSS puedes utilizar los combinadores de CSS **(>, +** y **~)** dentro de una anidación.

5.1. Anidación de propiedades

Sass permite anidar propiedades, además de selectores. Esta técnica agrupa declaraciones que comparten un prefijo, como **font-, margin**- o **border-.**

Propiedades de fuente

SCSS

```
.titulo {
  font: {
    family: Arial, sans-serif;
    size: 24px;
    weight: bold;
    style: italic;
    variant: small-caps;
  }
}
```

CSS GENERADO

```
.titulo {
  font-family: Arial, sans-serif;
  font-size: 24px;
  font-weight: bold;
  font-style: italic;
  font-variant: small-caps;
}
```

Sass combina el prefijo exterior (font) con cada nombre interior mediante un guion.

Propiedades de margen

SCSS

```scss
.caja {
  margin: {
    top: 10px;
    right: 20px;
    bottom: 15px;
    left: 20px;
  }
}
```

CSS GENERADO

```css
.caja {
  margin-top: 10px;
  margin-right: 20px;
  margin-bottom: 15px;
  margin-left: 20px;
}
```

Propiedades de borde

SCSS

```scss
.caja {
  border: {
    width: 2px;
    style: solid;
    color: red;
  }
}
```

CSS GENERADO

```css
.caja {
  border-width: 2px;
  border-style: solid;
  border-color: red;
}
```

5.2. El selector padre &

El símbolo & representa el selector padre actual. Es especialmente útil con pseudoclases, pseudoelementos y modificadores de clases.

SCSS

```scss
.input {
  background: #3498db;

  &:hover {
    background: #217dbb;
  }

  &-grande {
    padding: 1rem 2rem;
  }

  &[required] {
    border-left: 4px solid red;
  }

  &[type="email"] {
    background-color: lightblue;
  }

  &.destacado { border: 2px solid gold; }

}
```

CSS GENERADO

```css
.input {
  background: #3498db;
}

.input:hover {
  background: #217dbb;
}

.input-grande {
  padding: 1rem 2rem;
}

.input[required] {
  border-left: 4px solid red;
}
```

```
.input[type=email] {
  background-color: lightblue;
}

.input.destacado {
  border: 2px solid gold;
}
```

6. INTERPOLACIÓN

La interpolación inserta el valor de una expresión Sass dentro de un selector, nombre de propiedad o texto. Se escribe con **#{$variable}.**

```
$lado: left;
$nombre: aviso;

.#{$nombre} {
  border-#{$lado}: 4px solid #e67e22;
}
```

El selector generado será .aviso y la propiedad resultante será border-left.

POR QUÉ SE UTILIZA

La interpolación sirve cuando una variable forma parte de un nombre (clase, propiedad, etc.), no solo del valor de una declaración. Por ejemplo, si $estado: activo, el selector .boton-#{$estado} se convierte en .boton-activo.

7. LISTAS Y MAPAS

Sass dispone de tipos de datos más ricos que CSS. Las listas agrupan valores ordenados y los mapas almacenan parejas clave-valor. Los mapas son especialmente útiles para paletas de color, tamaños o configuraciones.

```
@use "sass:map";

$colores: (
  primario: #3498db,
  exito: #2ecc71,
  error: #e74c3c
);
```

```
.aviso {
  color: map.get($colores, error);
}
```

```
@use "sass:list";

$tamanos: 12px, 16px, 24px;

.texto {
  font-size: list.nth($tamanos, 2); // selecciona 16px
}
```

8. REUTILIZACIÓN DE ESTILOS

8.1. Mixins

Un mixin es un bloque reutilizable de declaraciones. Se define con @mixin y se inserta donde se necesita mediante @include.

SCSS

```
@mixin centrar-flex {
  display: flex;
  justify-content: center;
  align-items: center;
}

.cabecera { @include centrar-flex; }
.modal { @include centrar-flex; }
```

CSS GENERADO

```
.cabecera { display: flex; justify-content: center; align-items: center; }
.modal { display: flex; justify-content: center; align-items: center; }
```

8.2. Mixins con parámetros

Los mixins pueden recibir argumentos. También pueden definir valores por defecto, lo que permite crear componentes configurables sin repetir código.

```
@mixin boton($fondo, $texto: white) {
  background-color: $fondo;
  color: $texto;
  padding: .75rem 1.25rem;
  border: 0;
  border-radius: .4rem;
```

```
}

.boton-primario {
  @include boton(steelblue);
}

.boton-claro {
  @include boton(lightyellow, black);
}
```

EXPLICACIÓN

En @mixin boton($fondo, $texto: white), el color de fondo es obligatorio y el color de texto tiene un valor por defecto. @include boton(steelblue) usa texto blanco; @include boton(lightyellow, black) usa texto negro. Los parámetros permiten reutilizar una estructura cambiando solo algunos valores.

8.3. Funciones

Las funciones Sass calculan y devuelven un valor. Se definen con @function y devuelven el resultado mediante @return. A diferencia de un mixin, una función se utiliza dentro de una expresión.

SCSS

```
@use "sass:math";

@function px-a-rem($px, $base: 16px) {
  @return math.div($px, $base) * 1rem;
}

h1 {
  font-size: px-a-rem(32px);
}
```

CSS GENERADO

```
h1 { font-size: 2rem; }
```

DIFERENCIA ENTRE MIXIN Y FUNCIÓN

Un mixin inserta varias declaraciones CSS; una función devuelve un valor que puede utilizarse en una propiedad. Por ejemplo, px-a-rem(32px) devuelve 2rem si la base es 16px. Se utiliza así: font-size: px-a-rem(32px);

8.4. @extend y selectores placeholder

@extend permite indicar que un selector comparte los estilos de otro.

Sass permite extender tanto una clase normal (.mensaje-base) como un selector placeholder (%mensaje-base). La diferencia es que el placeholder no genera una regla CSS propia: solo aporta estilos cuando otro selector lo extiende.

1. @extend sin utilizar %

Podemos crear una clase base y extenderla desde otras clases.

SCSS

```scss
.mensaje-base {
  padding: 1rem;
  border-radius: 0.4rem;
  font-weight: bold;
}

.mensaje-exito {
  @extend .mensaje-base;
  background: lightgreen;
}

.mensaje-error {
  @extend .mensaje-base;
  background: lightcoral;
}
```

CSS GENERADO

```css
.mensaje-base,
.mensaje-exito,
.mensaje-error {
  padding: 1rem;
  border-radius: 0.4rem;
  font-weight: bold;
}


.mensaje-exito { background: lightgreen; }
.mensaje-error { background: lightcoral; }
```

Sass agrupa los selectores que comparten las propiedades. La clase .mensaje-base también aparece en el CSS, aunque no se utilice en el HTML.

2. @extend con un placeholder (%)

Si la base solo existe para compartir estilos, podemos definirla con %.

SCSS

```scss
%mensaje-base {
  padding: 1rem;
  border-radius: 0.4rem;
  font-weight: bold;
}

.mensaje-exito {
  @extend %mensaje-base;
  background: lightgreen;
}

.mensaje-error {
  @extend %mensaje-base;
  background: lightcoral;
}
```

CSS GENERADO

```css
.mensaje-exito,
.mensaje-error {
  padding: 1rem;
  border-radius: 0.4rem;
  font-weight: bold;
}

.mensaje-exito { background: lightgreen; }
.mensaje-error { background: lightcoral; }
```

El selector %mensaje-base no aparece en el CSS final. Si ningún selector lo extiende, Sass no genera ninguna regla para él.

9. ORGANIZACIÓN DE UN PROYECTO SASS

9.1. Partials

Cuando un proyecto crece, no conviene concentrar todo el SCSS en un único archivo. Sass permite dividir los estilos en archivos parciales o partials.

Un archivo parcial (partial) suele comenzar por guion bajo, como _variables.scss. Sass permite cargarlo escribiendo simplemente "variables", sin el guion ni la extensión.

```
scss/
  _variables.scss
  _mixins.scss
  estilos.scss
```

Los archivos parciales no suelen compilarse como hojas CSS independientes; se utilizan desde otros archivos. El archivo de entrada estilos.scss es el que compilamos.

9.2. @use: cargar y utilizar un módulo

@use carga otro archivo Sass y permite acceder a sus variables, mixins y funciones públicas. De forma predeterminada se usa un prefijo o espacio de nombres igual al nombre del archivo.

Archivo scss/_variables.scss:

```
$primario: #3498db;
$radio: 0.5rem;
$texto: #243447;
```

Archivo scss/estilos.scss:

```
@use "variables";

.boton {
  background: variables.$primario;
  border-radius: variables.$radio;
  color: variables.$texto;
}
```

CSS GENERADO

```
.boton {
  background: #3498db;
  border-radius: 0.5rem;
  color: #243447;
}
```

Observa que $primario se escribe variables.$primario: el prefijo indica de qué módulo procede. No se escribe simplemente $primario porque @use mantiene los nombres separados.

Cambiar el prefijo con as

```
@use "variables" as v;

.boton {
  background: v.$primario;
}
```

También existe @use "variables" as *; para acceder sin prefijo, pero en materiales iniciales conviene evitarlo porque facilita los conflictos de nombres.

Utilizar mixins de otro archivo

Archivo _mixins.scss:

```
@mixin redondeado($radio: 8px) {
  border-radius: $radio;
}
```

Archivo estilos.scss:

```
@use "mixins";

.tarjeta {
  @include mixins.redondeado(12px);
}
```

CSS GENERADO

```
.tarjeta {
  border-radius: 12px;
}
```

9.3. @forward: crear un punto de entrada común

@forward no se utiliza para aplicar directamente estilos en el archivo actual: su finalidad es volver a exponer variables, funciones y mixins de otros archivos. Así, el resto del proyecto puede cargar un solo módulo.

Estructura de archivos:

```
scss/
  tema/
    _variables.scss
    _mixins.scss
    _index.scss
  estilos.scss
```

Archivo tema/_variables.scss:

```
$texto: #243447;
$primario: #3498db;
```

Archivo tema/_mixins.scss:

```
@mixin redondeado {
  border-radius: 10px;
}
```

Archivo tema/_index.scss:

```
@forward "variables";
@forward "mixins";
```

Archivo estilos.scss:

```
@use "tema";

.boton {
  color: tema.$texto;
  background: tema.$primario;
  @include tema.redondeado;
}
```

CSS GENERADO

```
.boton {
  color: #243447;
  background: #3498db;
  border-radius: 10px;
}
```

Sass localiza tema/_index.scss al cargar "tema". Este archivo actúa como entrada pública: reúne los miembros de variables y mixins bajo un único prefijo, tema.

Diferencia esencial entre @use y @forward

Instrucción	¿Para qué sirve?	Ejemplo
@use	Utilizar los miembros de un módulo en el archivo actual	@use "variables";
@forward	Reexportar miembros para que otros archivos puedan utilizarlos	@forward "variables";

Importante: @forward por sí solo NO permite utilizar los miembros reenviados dentro del propio archivo. Si _index.scss necesitara utilizarlos también, tendría que incluir su correspondiente @use.

9.4. Módulos incorporados en Dart Sass

Dart Sass incluye bibliotecas de funciones ya preparadas. Se cargan con @use "sass:nombre" y se llaman con el prefijo correspondiente. No son archivos que tengamos que crear.

Módulo	Para qué se utiliza	Función representativa
sass:math	Cálculos matemáticos	math.div()
sass:color	Modificar componentes de colores	color.adjust()
sass:map	Consultar mapas clave-valor	map.get()
sass:list	Consultar listas	list.nth()
sass:string	Manipular textos	string.to-upper-case()
sass:meta	Consultar información sobre valores	meta.type-of()

Ejemplo 1. sass:math: dividir una anchura

```
@use "sass:math";

.columna {
  width: math.div(100%, 4);
}
```

CSS GENERADO

```
.columna {
  width: 25%;
}
```

math.div() realiza una división matemática durante la compilación. No debe confundirse con calc(), que permite al navegador resolver operaciones dependientes del tamaño disponible.

Ejemplo 2. sass:color: ajustar un color

```
@use "sass:color";

$base: #3498db;

.boton {
  background: $base;
  border-color: color.adjust($base, $lightness: -10%);
}
```

color.adjust() reduce aquí en diez puntos porcentuales la luminosidad HSL del color base. Sass genera un color calculado en el CSS final. Es útil para variantes de un mismo color; la transformación se decide en compilación.

Ejemplo 3. sass:map: obtener valores de un mapa

```
@use "sass:map";

$colores: (
  principal: #3498db,
  aviso: #e67e22
);

.alerta {
  color: map.get($colores, aviso);
}
```

CSS GENERADO

```
.alerta {
  color: #e67e22;
}
```

El mapa relaciona una clave (aviso) con un valor (#e67e22). map.get() devuelve el valor asociado a la clave.

Ejemplo 4. sass:list: acceder a una posición

```scss
@use "sass:list";

$tamanos: 12px, 16px, 24px;

.titulo {
  font-size: list.nth($tamanos, 3);
}
```

CSS GENERADO

```css
.titulo {
  font-size: 24px;
}
```

En Sass, las posiciones de las listas empiezan en 1, no en 0. Por eso el tercer elemento es 24px.

Ejemplo 5. sass:string y sass:meta

```scss
@use "sass:string";
@use "sass:meta";

$nombre: "bienvenido";
$medida: 20px;

.mensaje::before {
  content: string.to-upper-case($nombre);
}

@debug meta.type-of($medida);
```

CSS GENERADO

```css
.mensaje::before {
  content: "BIENVENIDO";
}
```

meta.type-of($medida) devuelve number y @debug lo muestra en la salida de compilación, no en el CSS generado. Es útil para comprobar tipos de datos mientras desarrollamos.

10. ESTRUCTURAS DE CONTROL

Las estructuras de control permiten decidir qué CSS se genera y repetir reglas de manera automática. Sass las evalúa durante la compilación: no reaccionan a los cambios del navegador en tiempo real.

10.1. Condicionales: @if, @else if y @else

Permiten elegir entre varias alternativas según una condición. @if comprueba la primera; @else if comprueba alternativas adicionales; @else se ejecuta si ninguna condición anterior se cumple. No es obligatorio escribir @else.

SCSS

```scss
@mixin tema($modo) {
  @if $modo == oscuro {
    background: #1f1f1f;
    color: white;
  } @else if $modo == suave {
    background: #eeeeee;
    color: #333333;
  } @else {
    background: white;
    color: black;
  }
}

body {
  @include tema(oscuro);
}
```

CSS GENERADO

```css
body {
  background: #1f1f1f;
  color: white;
}
```

La condición se comprueba una sola vez al compilar. Si cambiamos tema(oscuro) por tema(suave), se generará otra combinación de colores. No cambia automáticamente al pulsar un botón en la página.

10.2. Bucles: @each

@each recorre todos los elementos de una lista o las parejas clave-valor de un mapa. Es especialmente útil para crear clases de colores, tamaños o estados.

Recorrer una lista

SCSS

```scss
$tamanos: pequeno, mediano, grande;
@each $tamano in $tamanos {
  .boton-#{$tamano} {
```

```
    font-weight: bold;
  }
}
```

CSS GENERADO

```
.boton-pequeno { font-weight: bold; }
.boton-mediano { font-weight: bold; }
.boton-grande { font-weight: bold; }
```

La expresión #{$tamano} es una interpolación: inserta el valor de la variable dentro del nombre del selector.

Recorrer un mapa

SCSS

```
$colores: (
  primario: #3498db,
  exito: #2ecc71,
  error: #e74c3c
);
@each $nombre, $color in $colores {
  .texto-#{$nombre} {
    color: $color;
  }
}
```

CSS generado

```
.texto-primario { color: #3498db; }
.texto-exito { color: #2ecc71; }
.texto-error { color: #e74c3c; }
```

En cada vuelta, $nombre recibe la clave (primario, exito, error) y $color su valor correspondiente.

10.3. Bucles numéricos: @for

@for repite un bloque usando un contador. Con through se incluye el valor final; con to se excluye.

SCSS

```
@for $i from 1 through 4 {
  .m-#{$i} {
    margin: $i * 0.25rem;
  }
}
```

CSS GENERADO

```
.m-1 { margin: 0.25rem; }
.m-2 { margin: 0.5rem; }
.m-3 { margin: 0.75rem; }
.m-4 { margin: 1rem; }
```

Ejemplo con to

SCSS

```
@for $i from 1 to 4 {
  .col-#{$i} { width: $i * 25%; }
}
```

CSS GENERADO: no existe .col-4

```
.col-1 { width: 25%; }
.col-2 { width: 50%; }
.col-3 { width: 75%; }
```

10.4. Bucle condicional: @while

@while ejecuta un bloque mientras la condición sea verdadera. Se utiliza menos que @each y @for, porque es más fácil provocar un bucle infinito si olvidamos actualizar la variable.

SCSS

```
$i: 1;
@while $i <= 3 {
  .espacio-#{$i} {
    padding: $i * 8px;
  }
  $i: $i + 1;
}
```

CSS GENERADO

```
.espacio-1 { padding: 8px; }
.espacio-2 { padding: 16px; }
.espacio-3 { padding: 24px; }
```

En cada iteración aumenta $i en una unidad. Sin esa actualización, la condición seguiría siendo verdadera y el bucle no terminaría. Para intervalos conocidos suele ser más sencillo @for.

10.5. @debug, @warn y @error: mensajes de compilación

Estas directivas no son bucles ni condicionales. Sirven para diagnosticar problemas y validar datos. Sus mensajes aparecen en la terminal donde ejecutamos Sass, no en la web.

Directiva	Qué hace	¿Detiene la compilación?
@debug	Muestra información de depuración	No
@warn	Muestra una advertencia	No
@error	Indica un error que impide continuar	Sí

Ejemplo de @debug

SCSS

```scss
$ancho: 800px;
@debug $ancho;
.contenedor { width: $ancho; }
```

En la terminal aparece un mensaje de depuración con 800px. El CSS conserva únicamente .contenedor { width: 800px; }.

Ejemplo de @warn

SCSS

```scss
$columnas: 12;
@if $columnas > 6 {
  @warn "Se han definido demasiadas columnas";
}
.rejilla { display: grid; }
```

Se muestra una advertencia, pero se genera el CSS de .rejilla.

Ejemplo de @error

SCSS

```scss
$columnas: 0;
@if $columnas <= 0 {
  @error "El número de columnas debe ser mayor que cero";
}
.rejilla { display: grid; }
```

La compilación falla y no se obtiene una salida CSS válida para esta compilación.